

BE AIML SEM 7

SUBJECT: DEEP LEARNING
MODULE 2

COURSE CODE: CSC701
CREDIT 3



MODULE 2

TRAINING, OPTIMIZATION AND REGULARIZATION OF DEEP NEURAL NETWORK

Module 2 Syllabus:

2		Training, Optimization and Regularization of Deep Neural Network	10
	2.1	Training Feedforward DNN Multi Layered Feed Forward Neural Network, Learning Factors, Activation functions: Tanh, Logistic, Linear, Softmax, ReLU, Leaky ReLU, Loss functions: Squared Error loss, Cross Entropy, Choosing output function and loss function	
	2.2	Optimization Learning with backpropagation, Learning Parameters: Gradient Descent (GD), Stochastic and Mini Batch GD, Momentum Based GD, Nesterov Accelerated GD, AdaGrad, Adam, RMSProp	
	2.3	Regularization Overview of Overfitting, Types of biases, Bias Variance Tradeoff Regularization Methods: L1, L2 regularization, Parameter sharing, Dropout, Weight Decay, Batch normalization, Early stopping, Data Augmentation, Adding noise to input and output	

2.1. Training Feed-forward DNN

2.1

Training Feedforward DNN

Multi Layered Feed Forward Neural Network, Learning Factors, Activation functions: Tanh, Logistic, Linear, Softmax, ReLU, Leaky ReLU, Loss functions: Squared Error loss, Cross Entropy, Choosing output function and loss function

Topics Covered Under 2.1: Training Feed Forward Neural Network:

Multilayer Feed-Forward Neural Network (MFFNN)

- **Learning Factors**
- **Activation Functions**
- **Activation functions: Tanh, Logistic, Linear, Softmax, ReLU, Leaky ReLU**
- **Loss functions: Squared Error loss, Cross Entropy**
- **Choosing output function and loss function**

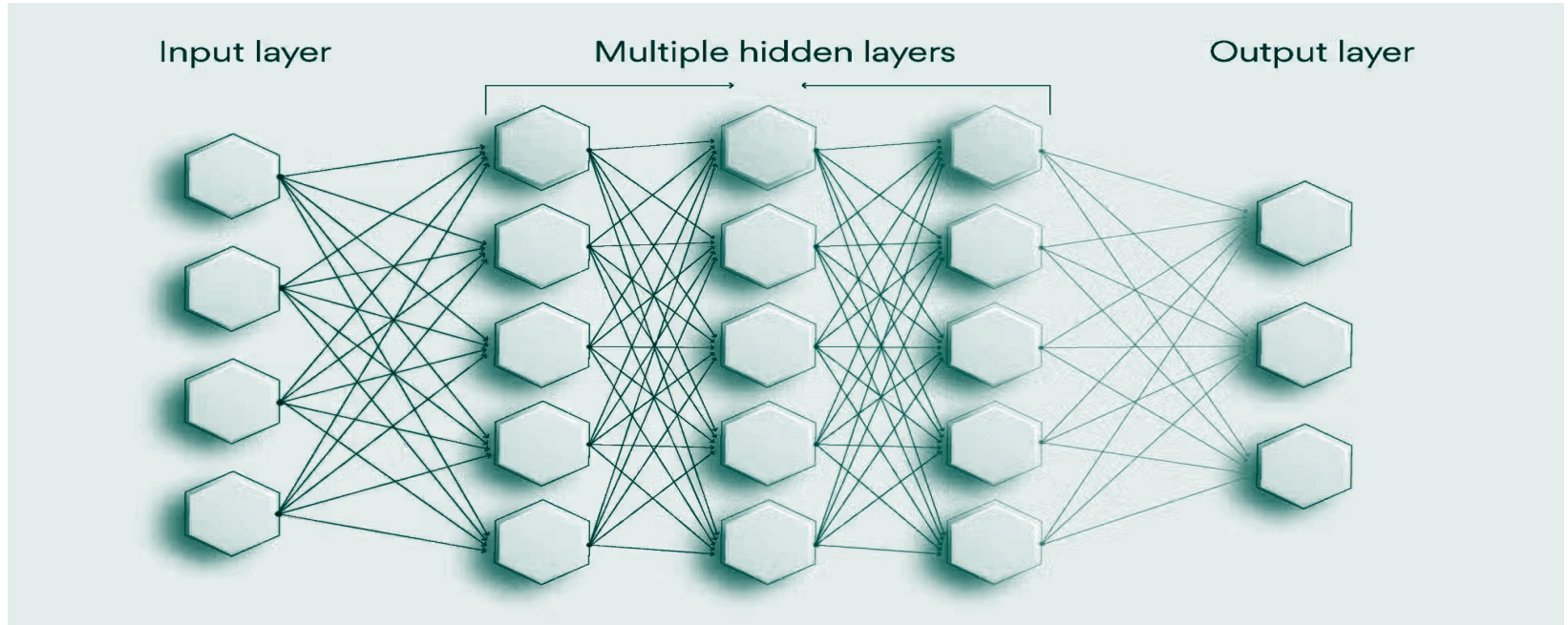
Multilayer Feed-Forward Neural Network (MFFNN)

- Multilayer Feed-Forward Neural Network (MFFNN) is an interconnected **Artificial Neural Network (ANN) with multiple layers** that has neurons with weights associated with them and they compute the result using activation functions.
- It is one of the types of Neural Networks in which the flow of the network is from input to output units and it does not have any loops, no feedback, and no signal moves in backward directions that is from output to hidden and input layer.

Working principle of (MFFNN)

- Each neuron in one layer is connected to every neuron in the next layer, making this a fully connected network.
- The strength of the connection between neurons is represented by weights, and learning in a neural network involves updating these weights based on the error of the output.
- The input and hidden layers use sigmoid and linear activation functions whereas the output layer uses a Heaviside step activation function at nodes because it is a two-step activation function that helps in predicting results as per requirements.
- All units also known as neurons have weights and calculation at the hidden layer is the summation of the dot product of all weights and their signals and finally the sigmoid function of the calculated sum.
- **Multiple hidden and output layer increases the accuracy of the output.**

Multilayer Feed-Forward Neural Network (MFFNN)



Advantages and Disadvantages of MFNN

- **Advantages/ Significance**

- Machine learning can be boosted with feed forward neural networks' simplified architecture.
- Multi-network in the feed forward networks operate independently, with a moderated intermediary.
- Complex tasks need several neurons in the network.
- Neural networks can handle and process nonlinear data easily compared to perceptrons and sigmoid neurons, which are otherwise complex.
- A neural network deals with the complicated problem of decision boundaries.
- Depending on the data, the neural network architecture can vary. For example, convolutional neural networks (CNNs) perform exceptionally well in image processing, whereas recurrent neural networks (RNNs) perform well in text and voice processing.

- **Disadvantages / Challenges:**

- Neural networks need graphics processing units (GPUs) to handle large datasets for massive computational and hardware performance. Several GPUs get used widely in the market, including Kaggle Notebooks and Google Collab Notebooks.

Learning Factors in MFNN

Training and convergence of the back propagation network is based on some important learning factors

Learning process in ANN mainly depends following factors:

- The number of layers in the network (Single-layered or multi-layered)
- Direction of signal flow (Feedforward or recurrent)
- the updation rule, the size and nature of the training set, and the architecture
- Initial weights
- Steepness of activation function
- Learning constant
- Momentum
- Network architecture (the updation rule, the size and nature of the training set, and the architecture)
- Necessary number of hidden neurons,
- Number of input neurons which is equal to the number of features of the input data set

SOME KEY LEARNING FACTORS

The factors are as follows:

- 1.** Initial weights
- 2.** Steepness of activation function
- 3.** Learning constant
- 4.** Momentum
- 5.** Network architecture
- 6.** Necessary number of hidden neurons

1. Initial weights:

The weights of the network to be trained are typically initialized at small random values.
The initialization strongly affects the ultimate solution

2. Steepness of activation function

The neuron's continuous activation function is characterized by its steepness factor
Also the derivative of the activation function serves as a multiplying factor in building components of the error signal vectors.

3. Learning constant:

The effectiveness and convergence of the error back propagation learning algorithm deepen significantly on the value of the learning constant.

4. Momentum:

The purpose of the momentum method is to accelerate the convergence of the error back propagation learning algorithm.

The method involves supplementing the current weight adjustment with a fraction of the most recent weight adjustment.

5. Network architecture:

One of the most important attributes if a layered neural network design is choosing the architecture

The number of input nodes is simply determined by the dimension or size of the input vector to be classified. The input vector size usually corresponds to the total number of distinct features of the input patterns.

6. Necessary number of hidden neurons:

This problem of choice of size of the hidden layer is under intensive study with no conclusive answers available.

One formula can be used to find out how many hidden layer neurons need to be used to achieve classification into M classes in x dimensional patterns space.

ACTIVATION FUNCTIONS

Linear

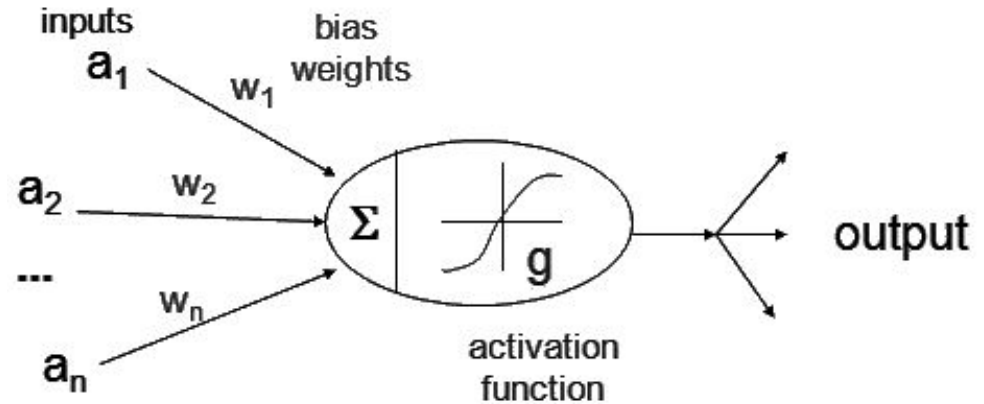
Logistic / Sigmoid

Tanh

ReLU

Leaky ReLU

Softmax

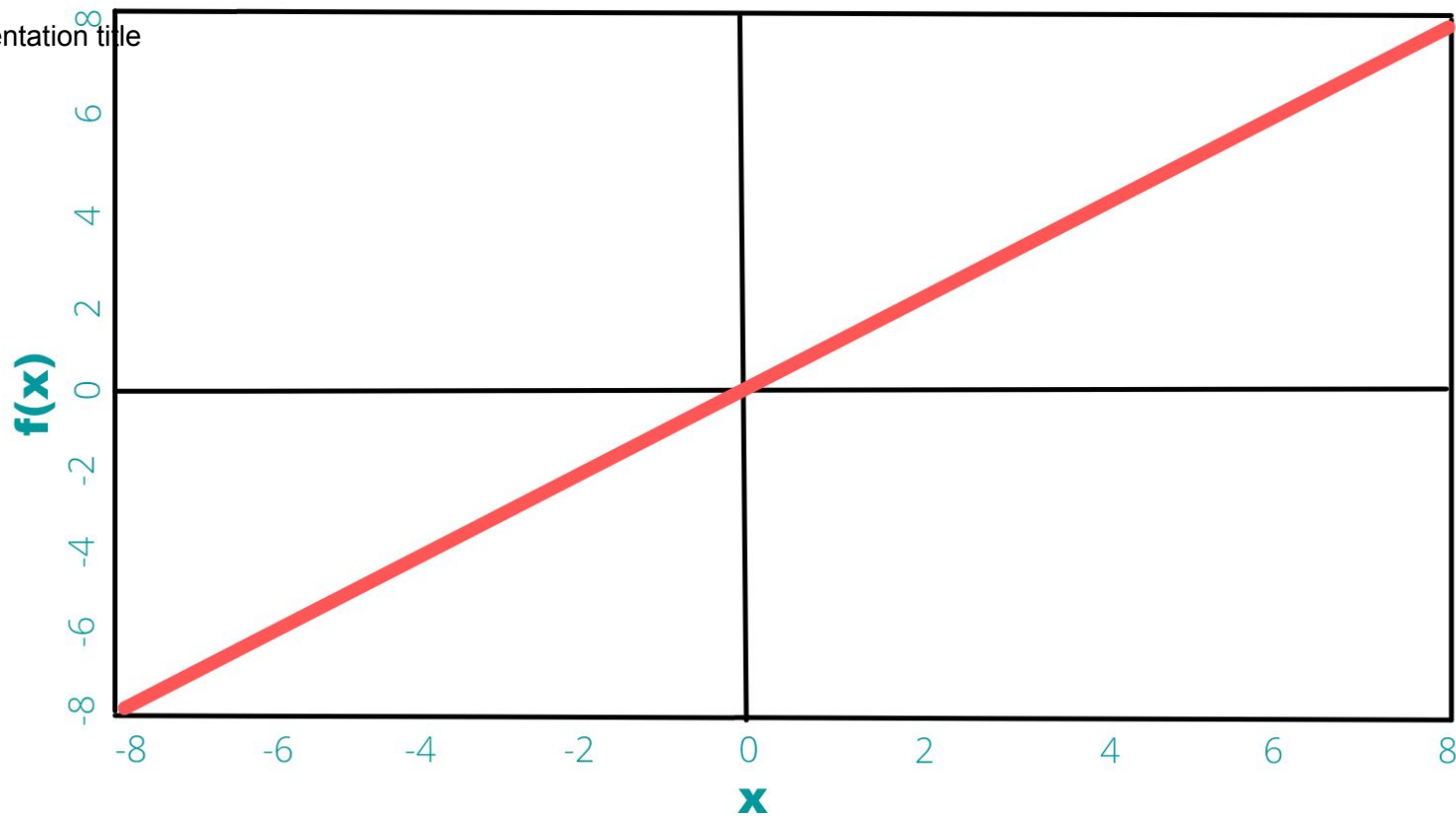


TYPES OF ACTIVATION FUNCTION

- Activation functions are generally two types, These are
1. Linear or Identity Activation Function
 2. Non-Linear Activation Function.

Generally, neural networks use **non-linear activation functions**, which can help the network learn complex data, compute and learn almost any function representing a question, and provide accurate predictions. They **allow back-propagation** because they have a derivative function which is related to the inputs.

Presentation title

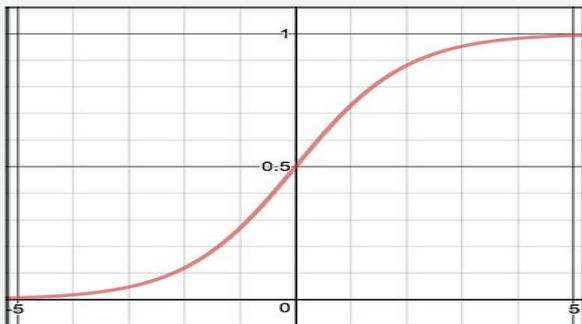


Linear Activation Function

- Sigmoid Activation function is very simple which takes a real value as input and gives probability that 's' always between 0 or 1. It looks like 'S' shape.
- Main advantage is simple and good for classifier. But Big disadvantage of the function is that it gives rise to a problem of “vanishing gradients” because its output isn't zero centered. It makes the gradient updates go too far in different directions. $0 < \text{output} < 1$, and it makes optimization harder. That takes very high computational time in hidden layer of neural network

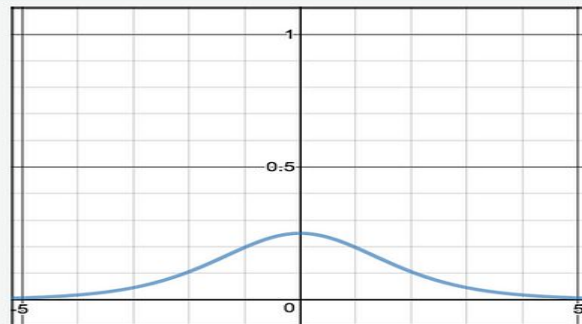
Function

$$S(z) = \frac{1}{1 + e^{-z}}$$



Derivative

$$S'(z) = S(z) \cdot (1 - S(z))$$



Sigmoid (Logistic) Function

The sigmoid function maps inputs to a range between 0 and 1:

$$S(x) = \frac{1}{1 + e^{-x}} = (1 + e^{-x})^{-1}$$

Derivation:

Apply the power rule and chain rule:

1. $\frac{d}{dx} (1 + e^{-x})^{-1} = -1(1 + e^{-x})^{-2} \cdot \frac{d}{dx} (1 + e^{-x})$

2. $\frac{d}{dx} (1 + e^{-x}) = -e^{-x}$

3. Combine terms: $-1(1 + e^{-x})^{-2} \cdot (-e^{-x}) = \frac{e^{-x}}{(1 + e^{-x})^2}$

4. Adjust fractions to match $S(x)$: $\frac{e^{-x}}{(1 + e^{-x})^2} = \frac{1}{1 + e^{-x}} \cdot \frac{e^{-x}}{1 + e^{-x}}$

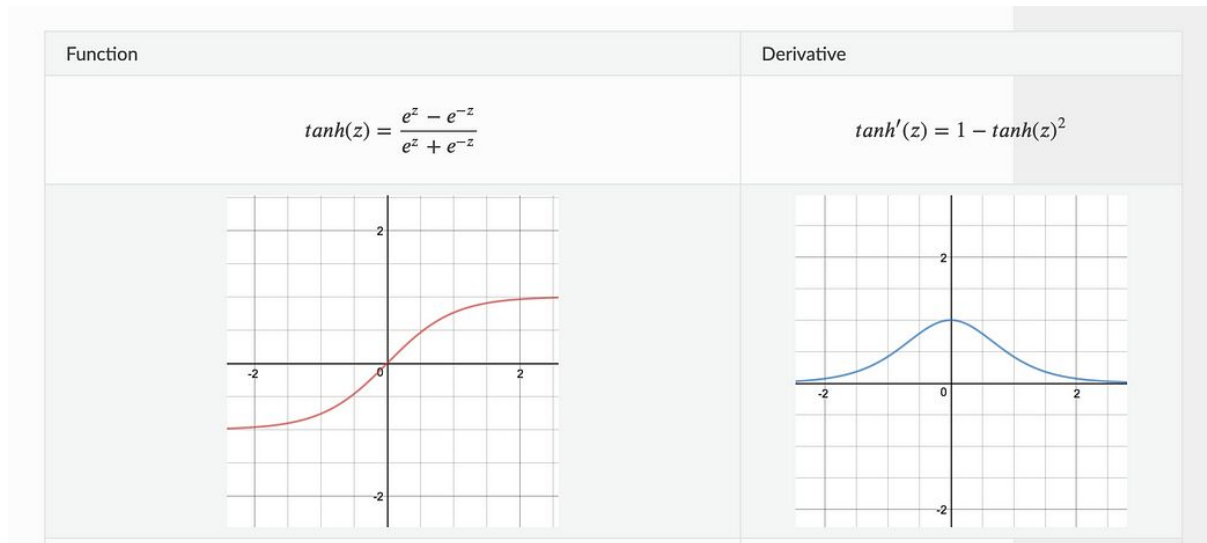
5. Add and subtract 1 in the numerator: $\frac{1}{1 + e^{-x}} \cdot \frac{(1 + e^{-x}) - 1}{1 + e^{-x}} = S(x) \cdot (1 - S(x))$ 

- **Derivative:** $S'(x) = S(x)(1 - S(x))$  YouTube · DeepLearningAI

Presentation title **Tanh or Hyperbolic tangent**

18

- Tanh help to solve non zero centered problem of sigmoid function. Tanh squashes a real-valued number to the range $[-1, 1]$. It's non-linear.
- Derivative function give us almost same as sigmoid's derivative function.
- It can't remove the vanishing gradient problem completely.



The tanh function squashes inputs between -1 and 1:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Derivation:

Apply the quotient rule $\left(\frac{u}{v} \right)' = \frac{u'v - uv'}{v^2}$:

1. Let $u = e^x - e^{-x} \implies u' = e^x + e^{-x}$

2. Let $v = e^x + e^{-x} \implies v' = e^x - e^{-x}$

3. Apply quotient rule: $\frac{(e^x + e^{-x})(e^x + e^{-x}) - (e^x - e^{-x})(e^x - e^{-x})}{(e^x + e^{-x})^2}$

4. Expand numerators: $\frac{(e^x + e^{-x})^2 - (e^x - e^{-x})^2}{(e^x + e^{-x})^2}$

5. Simplify the expression: $1 - \left(\frac{e^x - e^{-x}}{e^x + e^{-x}} \right)^2 = 1 - \tanh^2(x)$  YouTube · DeepLearningAI

• **Derivative:** $\tanh'(x) = 1 - \tanh^2(x)$  YouTube · DeepLearningAI

COMPARING TANH WITH SIGMOID

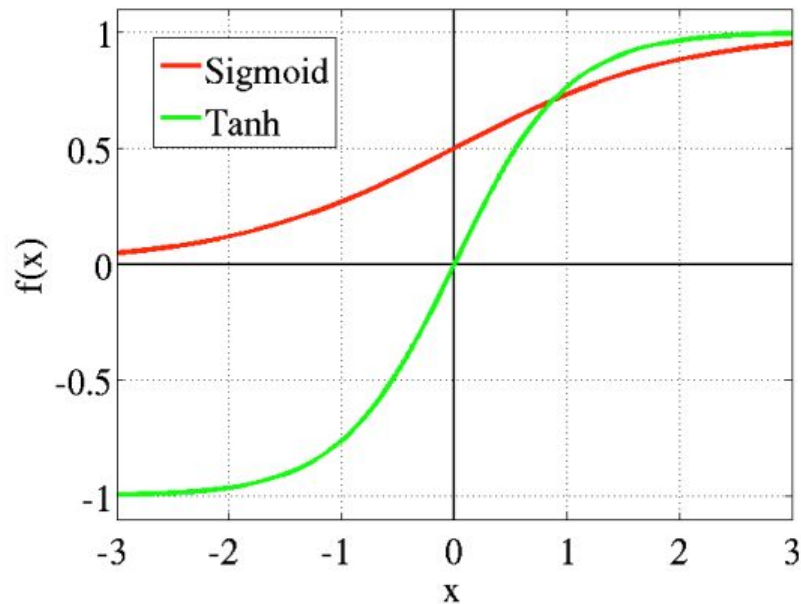


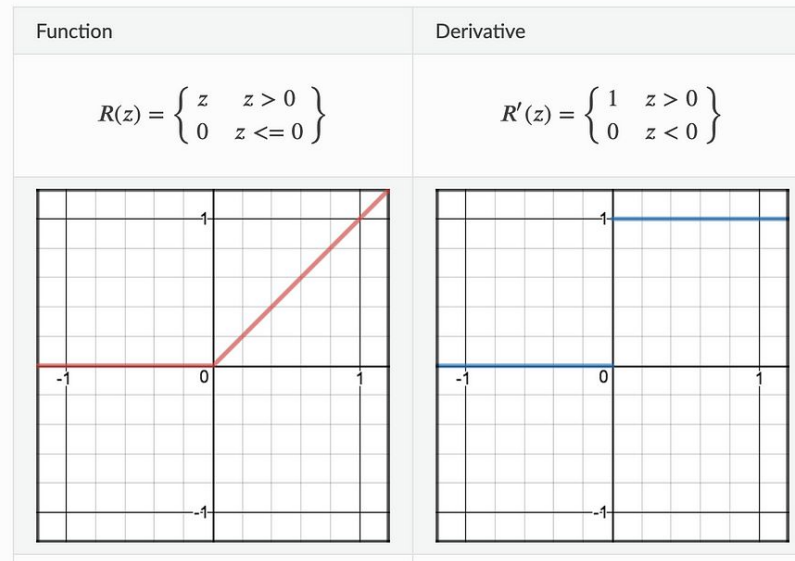
Fig: tanh v/s Logistic Sigmoid

RELU (Rectified Linear Unit)

This is most popular activation function which is used in hidden layer of NN. The formula is deceptively simple: $\max(0, z)$. Despite its name and appearance, it's not linear and provides the same benefits as Sigmoid but with better performance.

It's main advantage is that it avoids and rectifies vanishing gradient problem and less computationally expensive than tanh and sigmoid.

But it has also some draw back . Sometime some gradients can be fragile during training and can die. That leads to dead neurons.



Rectified Linear Unit (ReLU) Function

The ReLU function outputs the input directly if positive, and zero otherwise:

$$f(x) = \max(0, x)$$

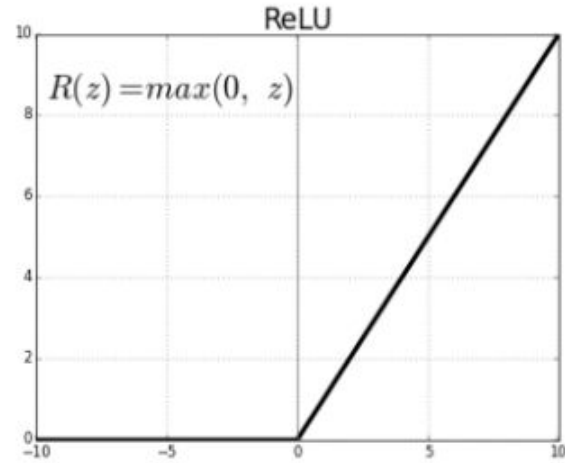
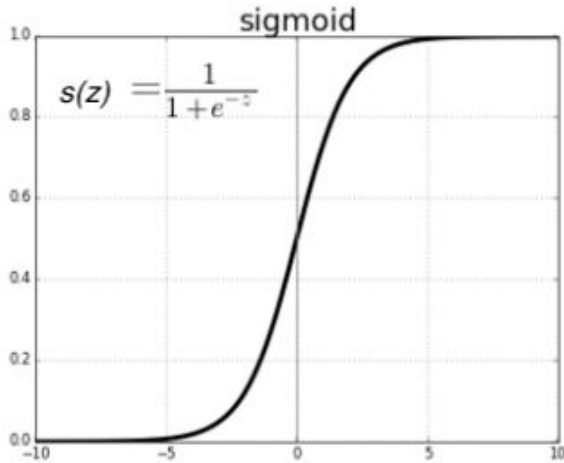
Derivation:

Differentiate piecewise for each domain:

- For $x > 0$, $f(x) = x \implies f'(x) = 1$
- For $x < 0$, $f(x) = 0 \implies f'(x) = 0$
- At $x = 0$, the derivative is technically undefined, set to 0 or 1 by convention.
- **Derivative:** $f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x < 0 \end{cases}$

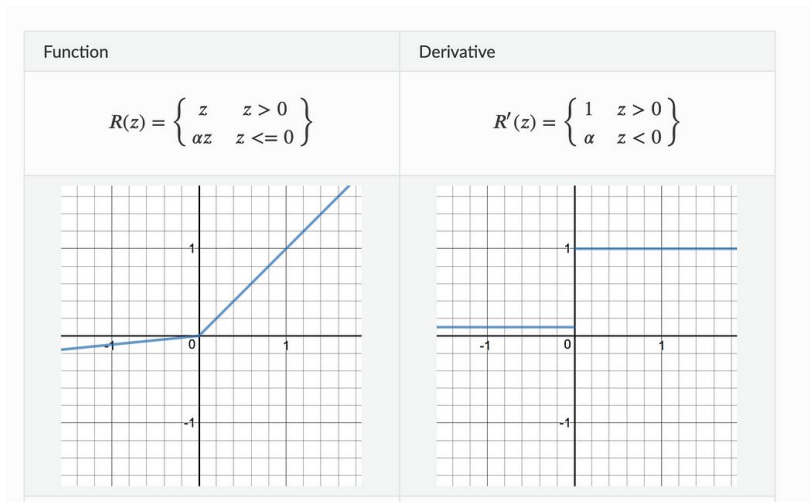


COMPARING RELU WITH SIGMOID



Sigmoid Vs ReLU

LEAKY RELU

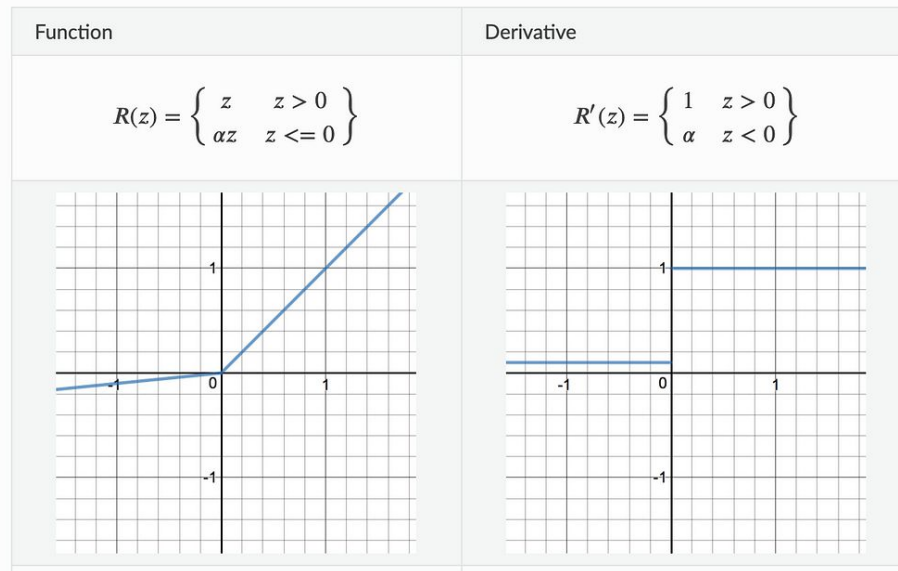


- It prevents dying ReLU problem. This variation of ReLU has a small positive slope in the negative area, so it does enable back-propagation, even for negative input values

RELU WITH LEAKY

RELU

Leaky ReLU provides consistent predictions for negative input values. During the front propagation if the learning rate is set very high it will overshoot killing the neuron.



Leaky ReLU Function

Leaky ReLU allows a small, non-zero gradient for negative inputs:

$$f(x) = \max(\alpha x, x)$$

where α is a small constant like 0.01.  YouTube · DeepLearningAI

Derivation:

Differentiate piecewise:

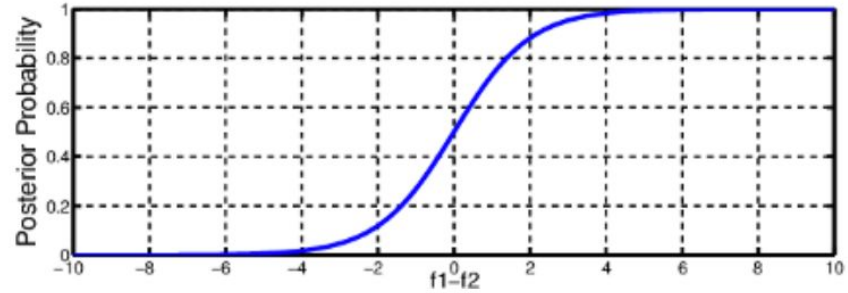
- For $x > 0$, $f(x) = x \implies f'(x) = 1$
- For $x < 0$, $f(x) = \alpha x \implies f'(x) = \alpha$

- **Derivative:** $f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ \alpha & \text{if } x < 0 \end{cases}$

 YouTube · DeepLearningAI

SOFTMAX

Generally, we use the function at last layer of neural network which calculates the probabilities distribution of the event over 'n' different events. The main advantage of the function is able to handle multiple classes.



SIGMOID WITH SOFTMAX

Let's take an example

Sigmoid input values: -0.5, 1.2, -0.1,
2.4

Sigmoid output values: 0.37, 0.77,
0.48, 0.91

SoftMax input values: -0.5, 1.2, -0.1, 2.4

SoftMax output values: 0.04, 0.21, 0.05,
0.70

$$f(x) = \text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

$$\text{softmax}(z_j) = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \text{ for } j = 1, \dots, K$$

Sigmoid's probabilities produced by a Sigmoid are independent. Furthermore, they are *not* constrained to sum to one: $0.37 + 0.77 + 0.48 + 0.91 = 2.53$. The reason for this is because the Sigmoid looks at each raw output value separately.

Whereas **Softmax's** the outputs are interrelated. The Softmax probabilities will always sum to one by design: $0.04 + 0.21 + 0.05 + 0.70 = 1.00$. In this case, if we want to increase the likelihood of one class, the other has to decrease by an equal amount.

LOSS AND COST FUNCTIONS FOR TRAINING DEEP LEARNING NEURAL NETWORKS

WHAT WE WILL STUDY

- Regression Models
 - Squared Error loss
- Classification Model
 - Cross Entropy
- Choosing output function and loss function

SQUARED ERROR LOSS

- In mathematical optimization and decision theory, a loss or cost function (sometimes also called an error function) is a function that maps an event or values of one or more variables onto a real number intuitively representing some “cost” associated with the event.
- In simple terms, the Loss function is a method of evaluating how well your algorithm is modeling your dataset. It is a mathematical function of the parameters of the machine learning algorithm.
- You can't improve what you can't measure. That's why the loss function comes into the picture to evaluate how well your algorithm is modeling your dataset.

Presentation title

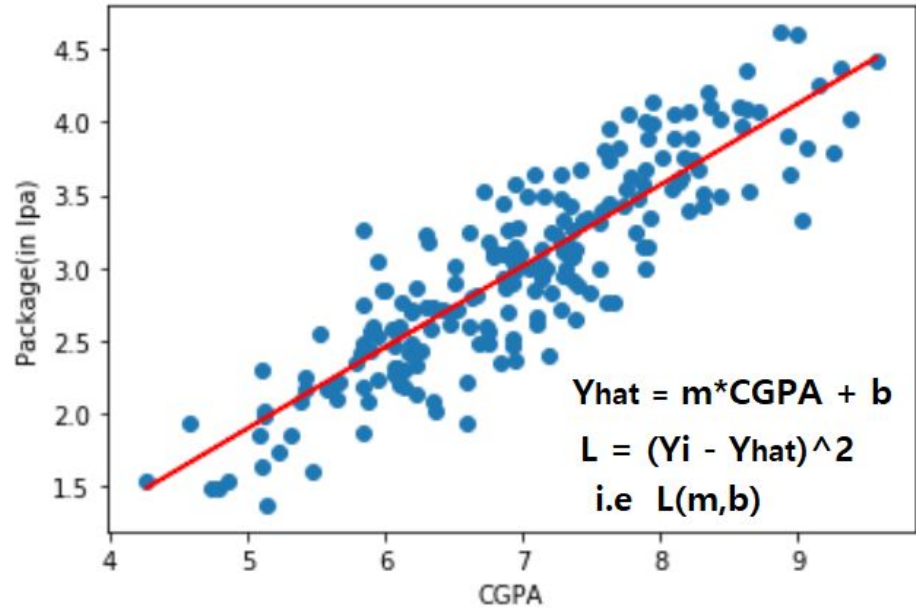
Advantage

1. Easy to interpret.
2. Always differential because of the square.
3. Only one local minima.

Disadvantage

1. Error unit in the square. because the unit in the square is not understood properly.
2. Not robust to outlier

Note - In regression at the last neuron use linear activation function.



$$\text{MSE} = \frac{1}{N} \sum_i^N (Y_i - \hat{Y}_i)^2$$

CROSS ENTROPY LOSS

- Binary

- It is used in binary classification problems like two classes. example a person has covid or not or my article gets popular or not.
- Binary cross entropy compares each of the predicted probabilities to the actual class output which can be either 0 or 1. It then calculates the score that penalizes the probabilities based on the distance from the expected value. That means how close or far from the actual value.

$$\text{Log Loss} = -\frac{1}{N} \sum_{i=1}^N y_i \log \hat{y}_i + (1-y_i) \log(1-\hat{y}_i)$$

- Categorical Cross Entropy

- Categorical Cross entropy is used for Multiclass classification and softmax regression.

$$\text{Loss} = - \sum_{j=1}^K y_j \log(\hat{y}_j)$$

where k is number of classes in the data



DIFFERENCE BETWEEN LOSS FUNCTION AND COST FUNCTION

- In deep learning, the loss function measures the error for a single training example, while the cost function is the average of the loss functions over the entire training dataset.
- Essentially, the loss function is a component of the cost function.
- The cost function guides the optimization process by providing a measure of overall model performance, and the loss function informs the optimization for individual data points.



WHAT IS LOSS FUNCTION?

- **Loss Function:**
 - **Calculates the error for a single data point (or a small batch of data points).**
 - **Examples include Mean Squared Error (MSE) for regression and Cross-Entropy loss for classification.**
 - **Provides feedback on how well the model is performing on specific examples.**
 - **Its value changes with each training example.**



WHAT IS COST FUNCTION?

Cost Function:

- **Calculates the average loss across the entire training dataset (or a batch).**
- **Often includes regularization terms to prevent overfitting.**
- **Provides a single number representing the overall performance of the model.**
- **Used by optimization algorithms (like gradient descent) to adjust model parameters.**



EXAMPLE FOR DIFFERENCE BETWEEN LOSS AND COST FUNCTION

- Imagine you have a dataset of images and want to train a model to identify cats.
- The loss function would calculate the error for each individual image prediction (e.g., how wrong the model was in identifying a cat).
- The cost function would then average all those individual image errors to give you a single number representing the overall performance of the model on the entire cat image dataset.



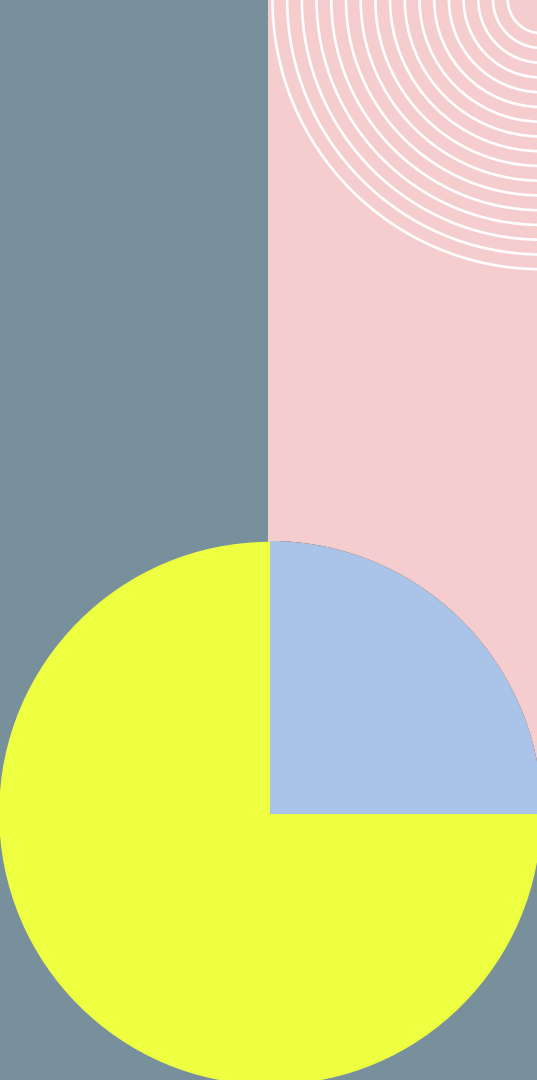
KEY DIFFERENCES SUMMARIZED

Feature	Loss Function	Cost Function
Scope	Single data point	Entire dataset
Calculation	Error for a single example	Average error over the dataset
Optimization	Used to compute individual errors during training	Used to compute overall error and guide optimization
Terminology	Sometimes used interchangeably with cost function, especially in neural networks	More generally refers to the overall error measurement

CHOOSING OUTPUT FUNCTION AND LOSS FUNCTION

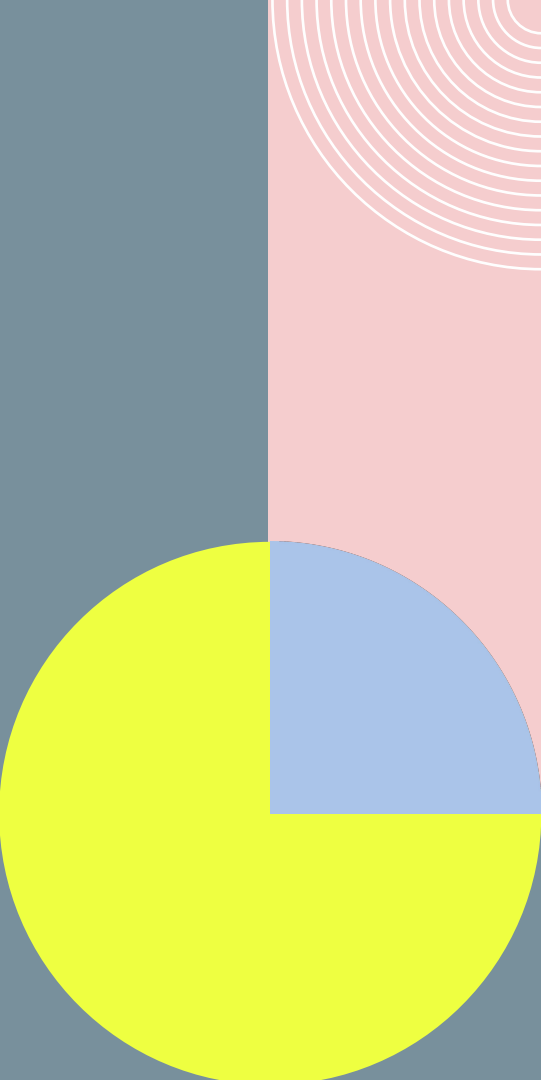
- The loss function calculates the error per observation, whilst the cost function calculates the error over the whole dataset.

Actual	Prediction	L1 loss
100,000	90,000	10,000
200,000	210,000	10,000
150,000	155,000	5,000
180,000	178,000	2,000
120,000	121,000	1,000



Choosing Output function and Loss Function in Deep Learning

- Choosing the right output and loss functions in deep learning is crucial for training effective models.
- The output function transforms the model's raw output into a usable format, while the loss function quantifies the error between predictions and ground truth, guiding the learning process.
- Selecting appropriate functions depends on the specific task (e.g., regression, classification) and data characteristics.



Output Functions:

❖ **Regression:**

For predicting continuous numerical values, a linear activation function is often used in the output layer.

❖ **Binary Classification:**

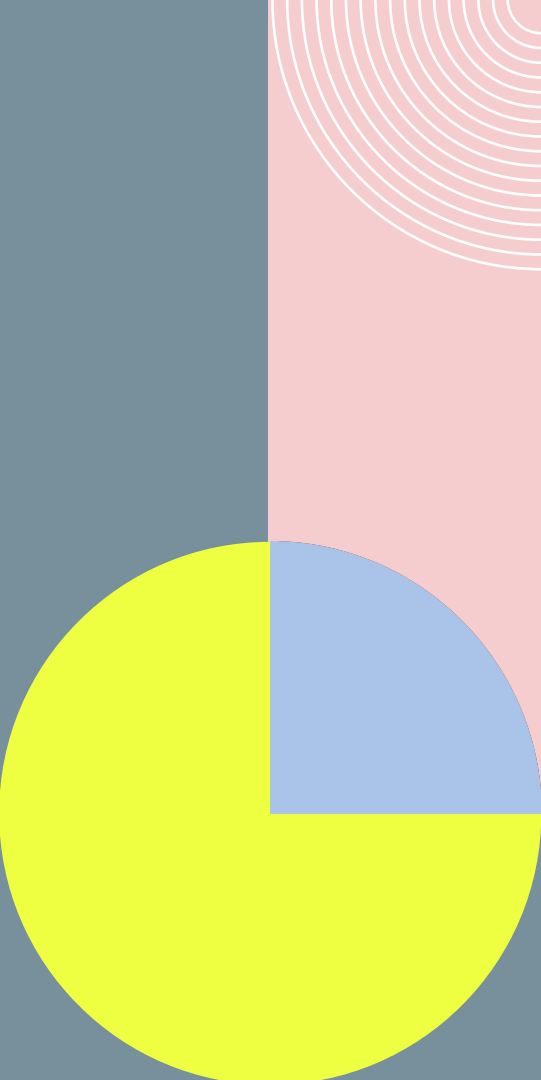
A sigmoid function is typically used, outputting a probability between 0 and 1, representing the likelihood of belonging to the positive class.

❖ **Multi-class Classification:**

A softmax function is commonly used, producing a probability distribution over multiple classes, ensuring that the probabilities sum to 1.

❖ **Sequence Generation:**

For tasks like machine translation or text generation, recurrent neural networks (RNNs) or transformers with appropriate output layers (e.g., using a vocabulary-based softmax for each time step) are employed



Loss Functions:

❖ **Mean Squared Error (MSE):**

Used for regression tasks, calculating the average squared difference between predicted and actual values.

❖ **Mean Absolute Error (MAE):**

Another regression loss, measuring the average absolute difference, more robust to outliers than MSE.

❖ **Binary Cross-Entropy:**

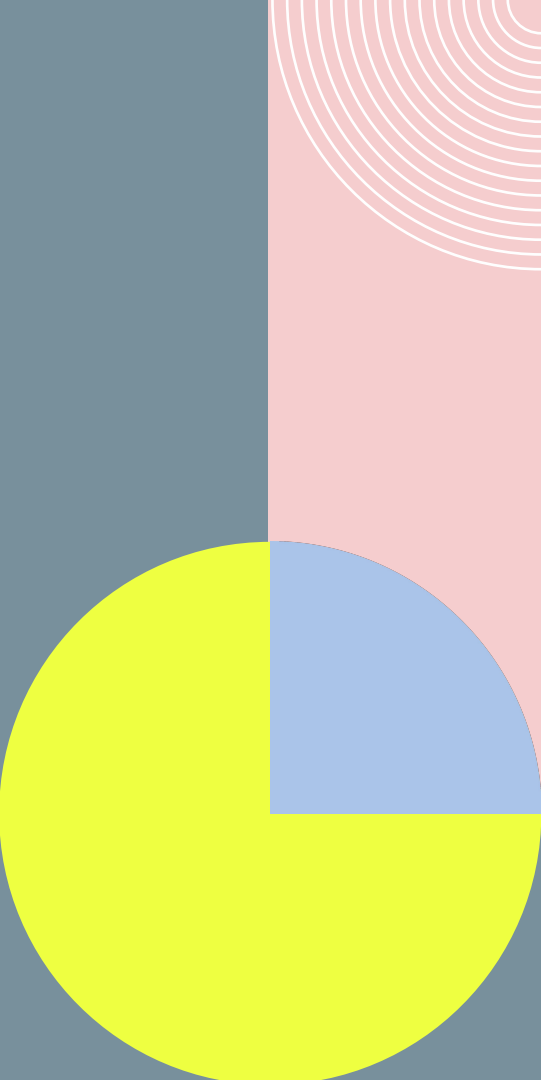
Used for binary classification, measuring the dissimilarity between predicted probabilities and true labels.

❖ **Categorical Cross-Entropy:**

Used for multi-class classification, measuring the dissimilarity between predicted probability distributions and true labels.

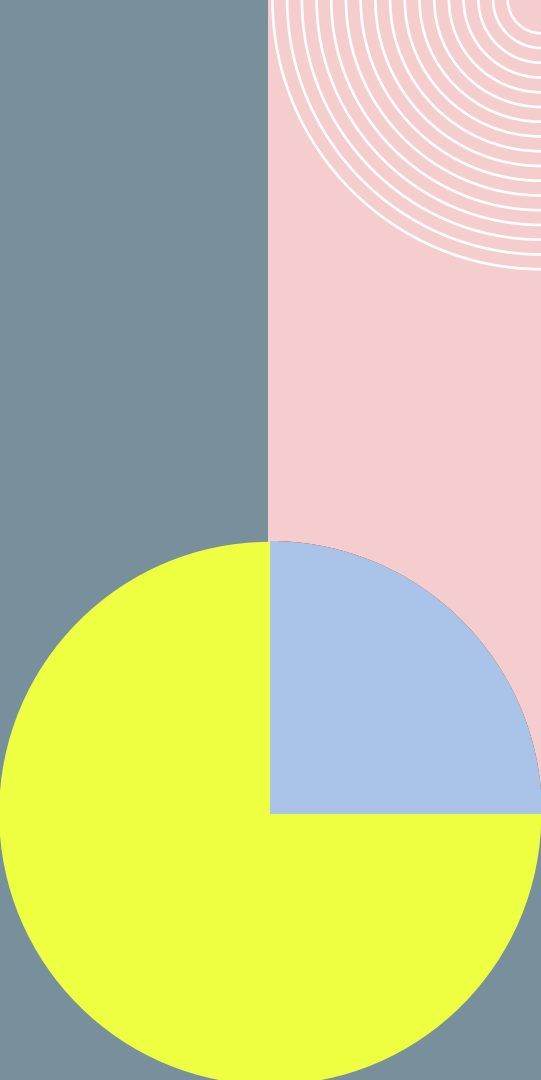
❖ **Hinge Loss:**

Commonly used in Support Vector Machines (SVMs) for binary classification, penalizing predictions that fall within a margin



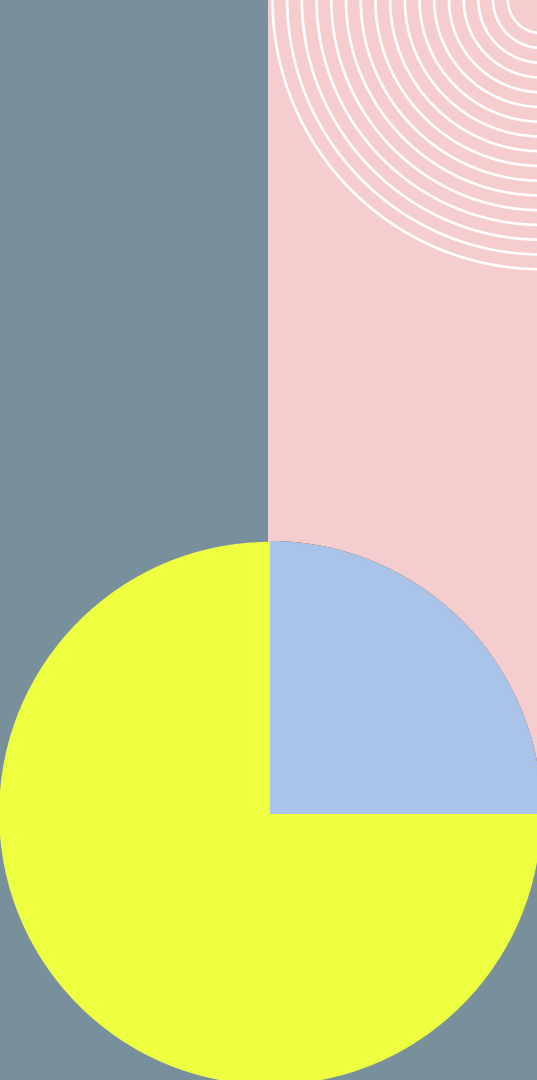
CONSIDERATIONS WHEN CHOOSING LOSS FUNCTION

- **Nature of the task:** Regression, classification (binary, multi-class), sequence generation, etc.
- **Data characteristics:** Presence of outliers, class imbalance, etc.
- **Model requirements:** Specific architectures or output types.



CHOOSING THE APPROPRIATE LOSS FUNCTION

- Loss functions are at the heart of deep learning, shaping how models learn and perform across diverse tasks.
- They are used to quantify the difference between predicted outputs and ground truth labels, guiding the optimization process to minimize errors.
- Selecting the right loss function is critical, as it directly impacts model convergence, generalization, and overall performance across various applications, from computer vision to time series forecasting.

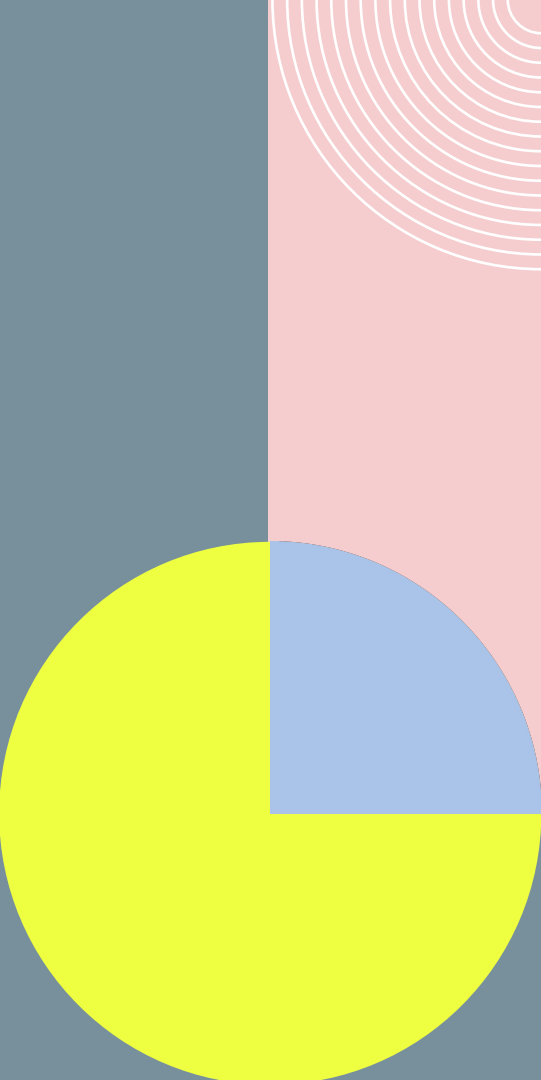


CHOOSING THE APPROPRIATE LOSS FUNCTION

Loss functions in deep learning play a critical role in optimizing neural networks during training by measuring the discrepancy between predicted outputs and actual ground truth labels

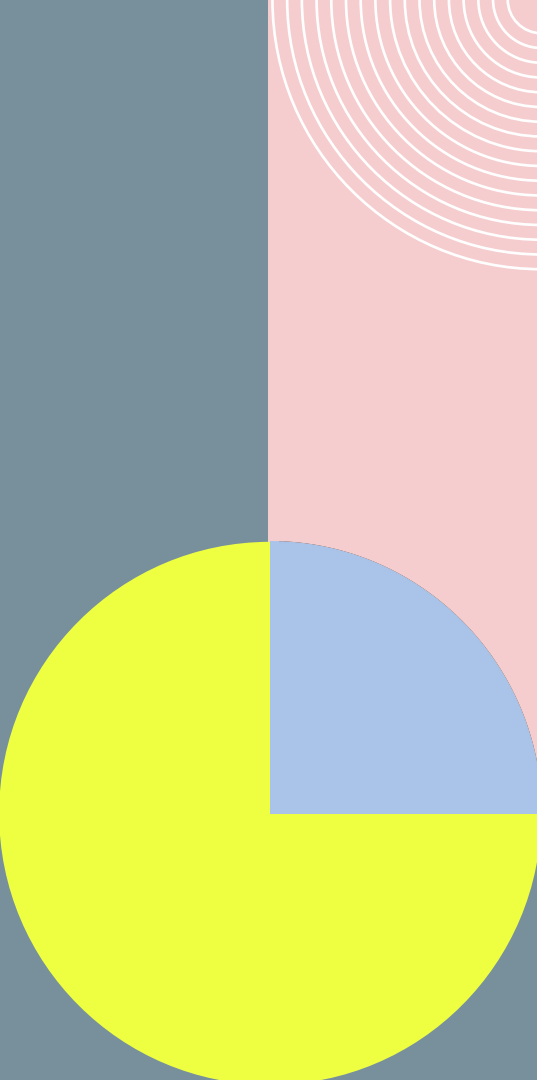
These functions are categorized into various types based on the task used. For example, for regression tasks involving continuous data, popular loss functions include Mean Squared Error (MSE) and Mean Absolute Error (MAE), which quantify the difference between predicted values and true labels.

On the other hand, for classification tasks dealing with categorical data, loss functions such as Binary Cross-Entropy and Categorical Cross-Entropy are commonly employed to calculate the error in predicted class probabilities [\[4\]](#).



CHOOSING THE APPROPRIATE LOSS FUNCTION

- In addition to standard loss functions, specialized loss functions like Huber Loss, Hinge Loss, and Dice Loss are utilized for specific tasks that require different forms of error measurement
- These custom loss functions cater to the unique requirements of tasks such as object detection, image segmentation, and natural language processing, where the standard loss functions may not be directly applicable.
- By selecting an appropriate loss function, deep learning models can effectively learn from data and optimize their parameters to improve predictive performance
- The choice of a suitable loss function in deep learning models has a significant impact on the training process and ultimately the model's accuracy
- A well-chosen loss function guides the optimization algorithm in the right direction, helping the model converge to a solution that minimizes the prediction error



CHOOSING THE APPROPRIATE OUTPUT AND LOSS FUNCTION

- In essence, the output function is closely tied to the nature of the prediction task, while the loss function guides the learning process by quantifying prediction errors. Careful consideration of both is vital for successful deep learning model training

SUMMARY OF 2.1

Type of Neural Network – DLL, CNN, RNN

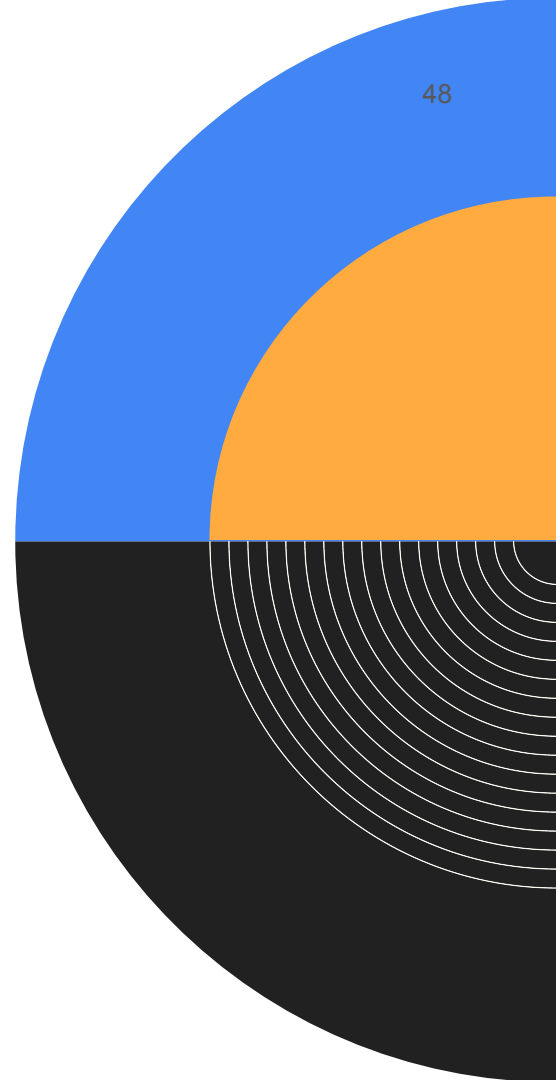
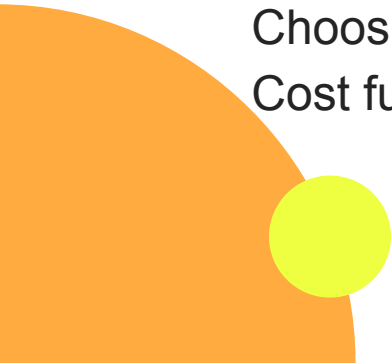
Gradient descent

Learning factors

Activation function – Linear, Sigmoid, Tanh, ReLu

Choosing the appropriate Loss and

Cost function



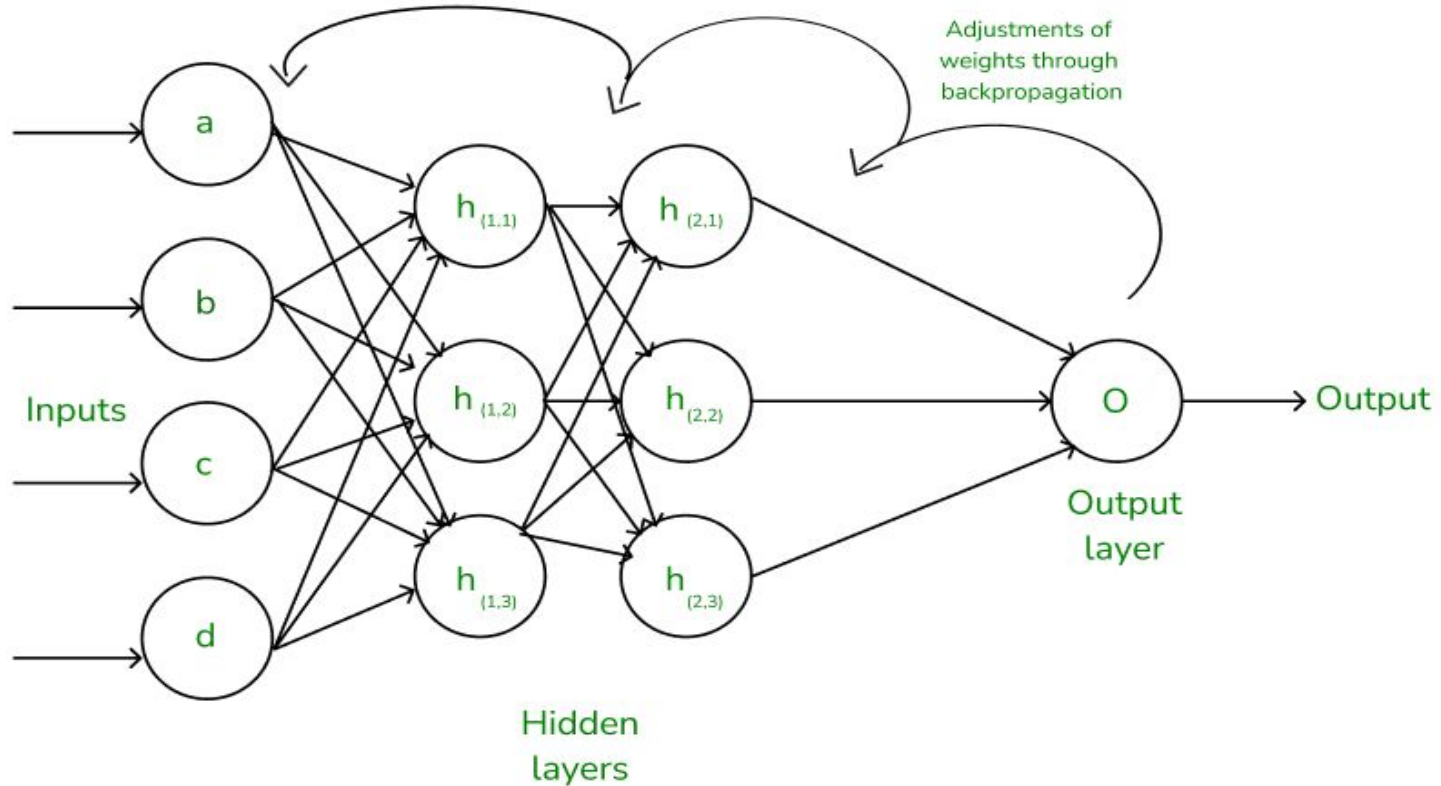
2.2. Optimization

2.2	Optimization Learning with backpropagation, Learning Parameters: Gradient Descent (GD), Stochastic and Mini Batch GD, Momentum Based GD, Nesterov Accelerated GD, AdaGrad, Adam, RMSProp
-----	--

Topics covered under 2.2 Optimization

- Learning with Backpropagation
- Learning Parameters:
- Gradient Descent (GD),
- Stochastic and Mini Batch Gradient descent (GD)
- Momentum Based Gradient Descent (GD)
- Nesterov Accelerated Gradient Descent (GD)
- AdaGrad
- Adam
- RMSProp

LEARNING WITH BACKPROPAGATION



WHAT IS LEARNING WITH BACKPROPAGATION?

- Backpropagation is an effective algorithm used to train artificial neural networks, especially in feed-forward neural networks.
- Backpropagation is an iterative algorithm, that helps to minimize the cost function by determining which weights and biases should be adjusted. During every epoch, the model learns by adapting the weights and biases to minimize the loss by moving down toward the gradient of the error. Thus, it involves the two most popular optimization algorithms, such as [gradient descent](#) or [stochastic gradient descent](#).
- Computing the gradient in the backpropagation algorithm helps to minimize the [cost function](#) and it can be implemented by using the mathematical rule called chain rule from calculus to navigate through complex layers of the neural network.

LEARNING PARAMETERS

- Gradient Descent (GD) Variants

- Stochastic
- Batch
- Mini Batch GD

- GD Optimization Algorithms

- Momentum Based GD
- Nesterov Accelerated GD
- AdaGrad
- Adam
- RMSProp

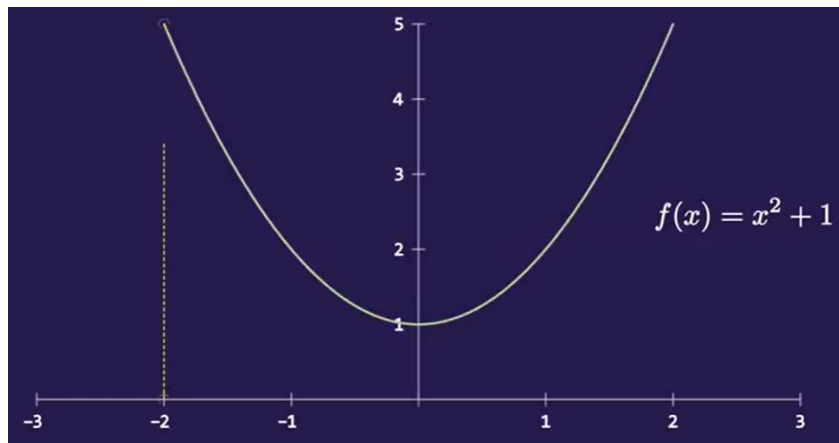
[CS231n Convolutional Neural Networks for Visual Recognition
deeplearningbook.org/contents/optimization.html](https://www.cs231n.org/2024/contents/optimization.html)

[An overview of gradient descent optimization algorithms \(ruder.io\)](https://www.ruder.io/gradient-descent/)

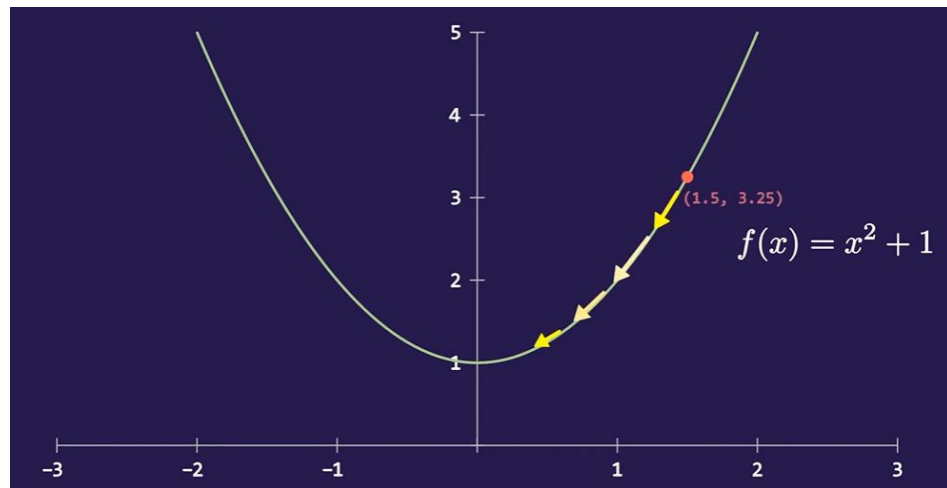
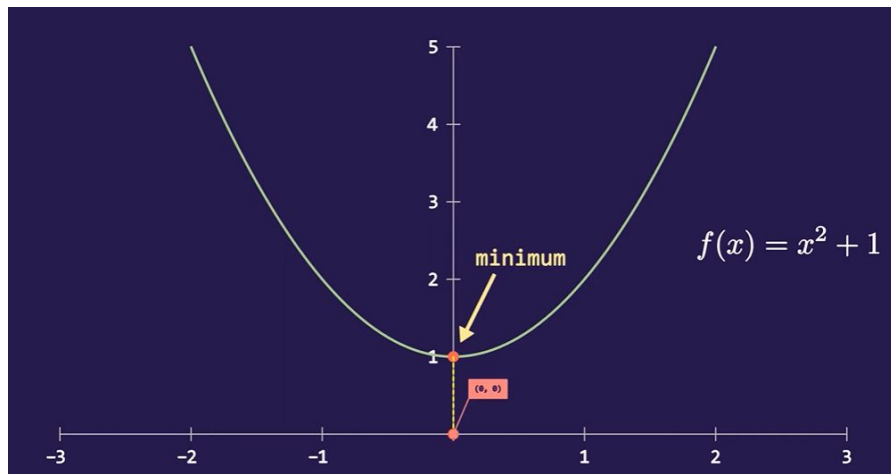
GRADIENT DESCENT (GD)

- We have learned – how to reduce error by using loss function
- To reduce the error DL uses Gradient Descent
- Now, question is what is GD?
- For given function finding the maximum or minimum values of that function.
- Let's take an example

$$f(x) = x^2 + 1$$



- Let's assume we are trying to find minimum of this function
- But not every function can be plotted as graph (3-D, 4-D etc)
- So, to find minimum of this graph we can initialize a point randomly anywhere in the graph and then find the direction in which this value is reducing fast, **this is called gradient, it can be computed by taking derivative of the function w.r.t. input parameters in our case x.**
- Using this gradient and learning rate, we can find the new value of x which will minimize the function



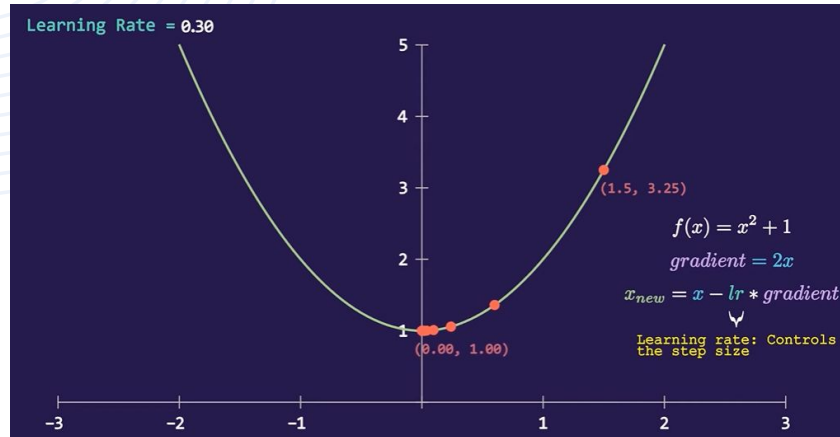
$$f(x) = x^2 + 1$$

$$\frac{\partial f}{\partial x} = 2x$$

$$f(x) = x^2 + 1$$

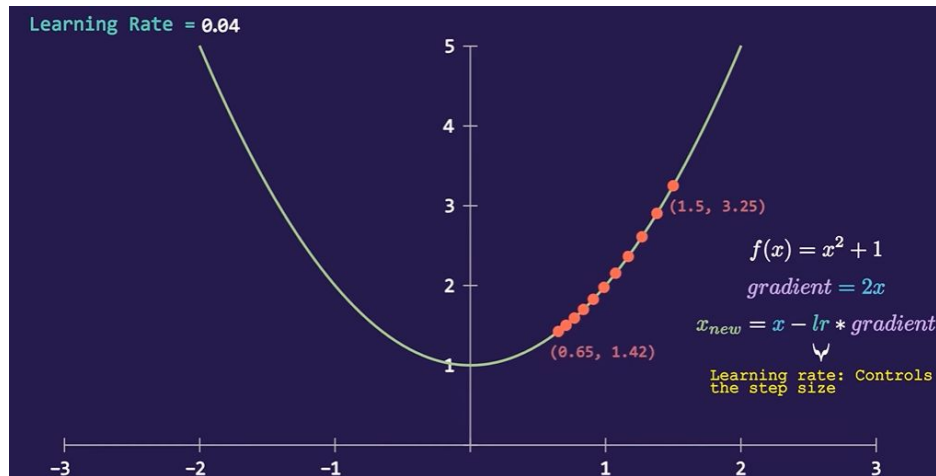
$$\text{gradient} = 2x$$

$$x_{\text{new}} = x - lr * \text{gradient}$$



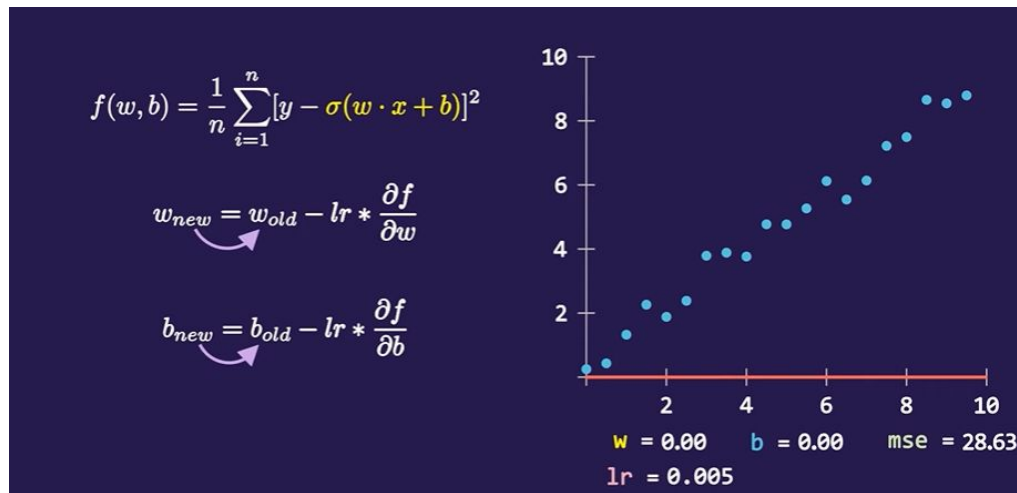
Effect of learning rate –
larger values may overshoot by skipping minima

Effect of learning rate –
Smaller values may take more steps to reach minima



GRADIENT DESCENT IN DEEP LEARNING

- In DL weights and bias are the only two learnable parameters (we have studied this in backpropagation)
- Hence, we have to optimize these two parameters to reduce the error made by our model
- Lets take an example of linear regression problem



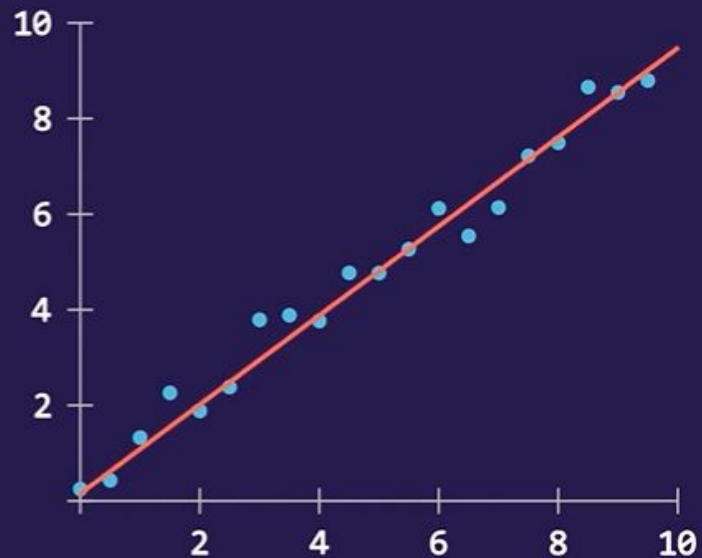
- The loss function is MSE
 $\frac{1}{n} (y_{\text{actual}} - y_{\text{pred}})^2$
- Now as we have two parameters we need to find gradients
- New values of weights and bias to find the particular value of w and b to reduce error
- Assume we are starting with $w=b=0$, initial line is as shown

- As we start gradient descent the mse will drop and the line will start moving and try to fit towards s_{58} data

$$f(w, b) = \frac{1}{n} \sum_{i=1}^n [y - \sigma(w \cdot x + b)]^2$$

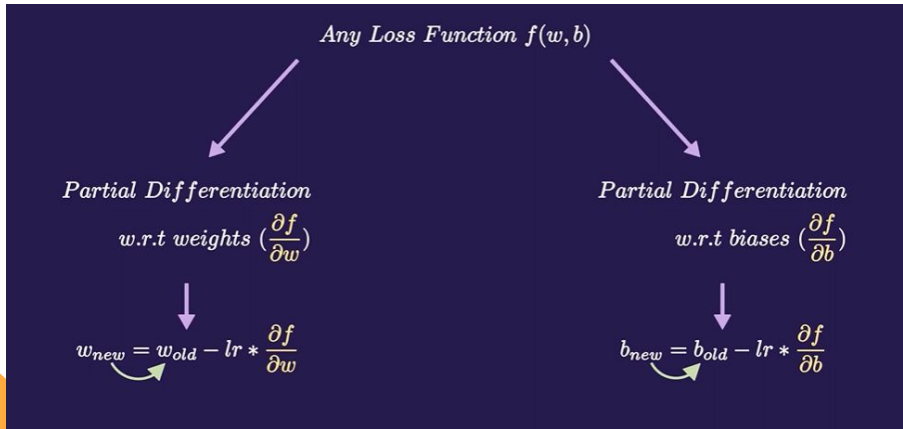
$$w_{new} = w_{old} - lr * \frac{\partial f}{\partial w}$$

$$b_{new} = b_{old} - lr * \frac{\partial f}{\partial b}$$



$w = 0.93$ $b = 0.15$ $mse = 0.15$
 $lr = 0.005$

GENERALIZED WAY OF CALCULATING GD



TYPES OF GRADIENT DESCENT

- Stochastic
- Batch
- Mini Batch GD

STOCHASTIC GD

$$\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i)}; y^{(i)}).$$

In the case of Stochastic Gradient Descent, we update the parameters after every single observation and we know that every time the weights are updated it is known as an iteration.

Class 1	Class 2
2	8
3	9
4	10
5	11
6	12
7	13

MINI BATCH GD

- 2 batch size in each epoch – ideal batch size 30 to 120

$$\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i:i+n)}; y^{(i:i+n)}).$$

Class 1	Class 2
2	8
3	9
4	10
5	11
6	12
7	13

BATCH GD

- For each epoch we train on entire dataset at once

$$\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta).$$

Class 1	Class 2
2	8
3	9
4	10
5	11
6	12
7	13

Minibatch sizes are generally driven by the following factors:

- Larger batches provide a more accurate estimate of the gradient, but with less than linear returns.
- Multicore architectures are usually underutilized by extremely small batches. This motivates using some absolute minimum batch size, below which there is no reduction in the time to process a minibatch.
- If all examples in the batch are to be processed in parallel (as is typically the case), then the amount of memory scales with the batch size. For many hardware setups this is the limiting factor in batch size.
- Some kinds of hardware achieve better runtime with specific sizes of arrays. Especially when using GPUs, it is common for power of 2 batch sizes to offer better runtime. Typical power of 2 batch sizes range from 32 to 256, with 16 sometimes being attempted for large models.
- Small batches can offer a regularizing effect ([Wilson and Martinez, 2003](#)), perhaps due to the noise they add to the learning process. Generalization error is often best for a batch size of 1. Training with such a small batch size might require a small learning rate to maintain stability because of the high variance in the estimate of the gradient. The total runtime can be very high as a result of the need to make more steps, both because of the reduced learning rate and because it takes more steps to observe the entire training set.

COMPARISON OF GD ALGORITHMS

Batch Gradient Descent	Stochastic Gradient Descent	Mini-Batch Gradient Descent
Since the entire training data is considered before taking a step in the direction of gradient, therefore it takes a lot of time for making a single update.	Since only a single training example is considered before taking a step in the direction of gradient, we are forced to loop over the training set and thus cannot exploit the speed associated with vectorizing the code.	Since a subset of training examples is considered, it can make quick updates in the model parameters and can also exploit the speed associated with vectorizing the code.
It makes smooth	It makes very noisy updates	Depending upon the batch size, the updates

GD OPTIMIZATION ALGORITHMS

- Momentum Based GD
- Nesterov Accelerated GD
- AdaGrad
- Adam
- RMSProp

MOMENTUM

$$change_{new} = lr * gradient + momentum * change_{old}$$

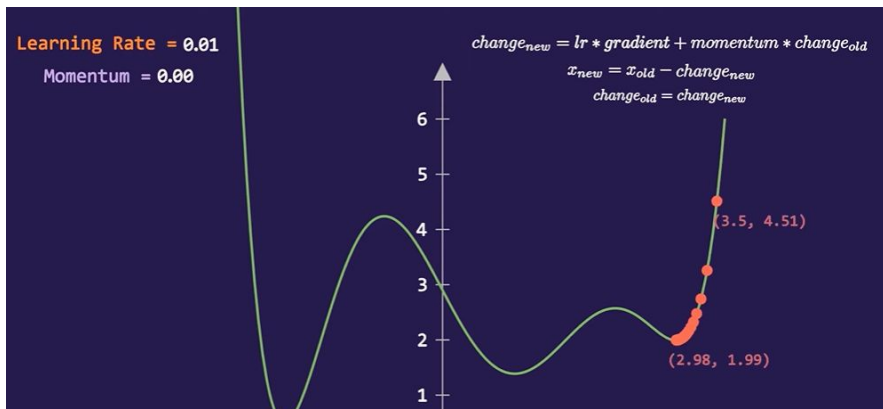
$$x_{new} = x_{old} - change_{new}$$

$$change_{old} = change_{new}$$

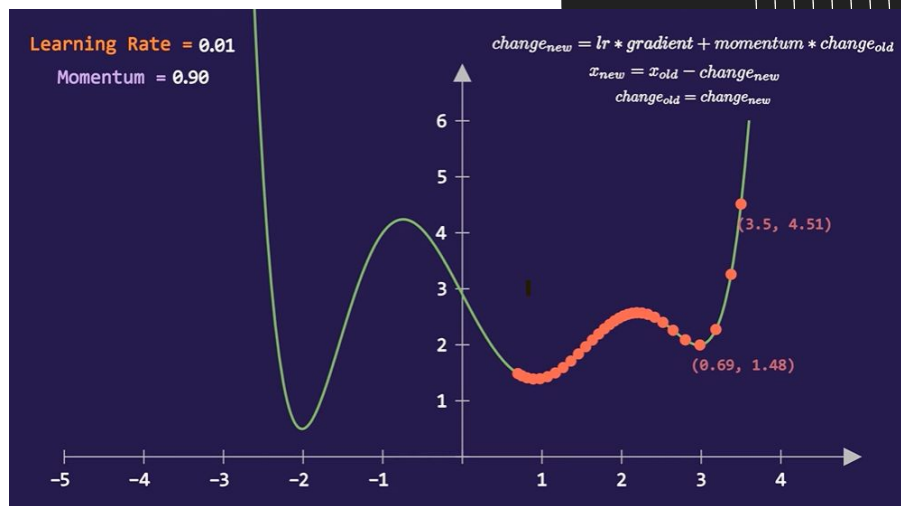
$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta)$$

$$\theta = \theta - v_t$$

WHY WE NEED MOMENTUM



Momentum is like a ball rolling down the hill, without momentum it may stuck in local minima



NESTEROV ACCELERATED GD

- There are few challenges with momentum based, oscillations- it overshoots its objective with lots of turns
- Look before you leap

So we know that we are going to move by at least by $\gamma \cdot \text{update}_{t-1}$ and then a bit more by $\eta \nabla w_t$

Why not calculate the gradient ($\nabla w_{\text{look_ahead}}$) at this partially updated value of w ($w_{\text{look_ahead}} = w_t - \gamma \cdot \text{update}_{t-1}$) instead of calculating it using the current value w_t

$$\begin{aligned}v_t &= \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta - \gamma v_{t-1}) \\ \theta &= \theta - v_t\end{aligned}$$

Update rule for NAG

$$\begin{aligned}w_{\text{look_ahead}} &= w_t - \gamma \cdot \text{update}_{t-1} \\ \text{update}_t &= \gamma \cdot \text{update}_{t-1} + \eta \nabla w_{\text{look_ahead}} \\ w_{t+1} &= w_t - \text{update}_t\end{aligned}$$

We will have similar update rule for b_t

ADAGRAD

- It adapts the learning rate to the parameters, performing smaller updates (i.e. low learning rates) for parameters associated with frequently occurring features, and larger updates (i.e. high learning rates) for parameters associated with infrequent features. For this reason, it is well-suited for dealing with sparse data.

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} \cdot g_{t,i}.$$

ADAM

- Adaptive Moment Estimation (Adam)
- In addition to storing an exponentially decaying average of past squared gradients v_t like Adadelta and RMSprop, Adam also keeps an exponentially decaying average of past gradients m_t , similar to momentum. Whereas momentum can be seen as a ball running down a slope, Adam behaves like a heavy ball with friction, which thus prefers flat minima in the error surface

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

2.3. Regularization

2.3 Regularization

Overview of Overfitting, Types of biases, Bias Variance Tradeoff
Regularization Methods: L1, L2 regularization, Parameter sharing, Dropout, Weight Decay, Batch normalization, Early stopping, Data Augmentation, Adding noise to input and output

Topics covered under 2.3 Regularization

- Overview of Overfitting, Types of biases, Bias Variance Tradeoff
- Regularization Methods: L1, L2 regularization, Parameter sharing, Dropout, Weight Decay, Batch normalization, Early stopping, Data Augmentation, Adding noise to input and output

OVERFITTING

- A model with high Variance will have a tendency to be overly complex. This causes the overfitting of the model. Low Testing Accuracy: A model with high Variance will have very high training accuracy (or very low training loss), but it will have a low testing accuracy (or a low testing loss)

BIAS AND VARIANCE

- Whenever we discuss model prediction, it's important to understand prediction errors (bias and variance). There is a tradeoff between a model's ability to minimize bias and variance. Gaining a proper understanding of these errors would help us not only to build accurate models but also to avoid the mistake of overfitting and underfitting.

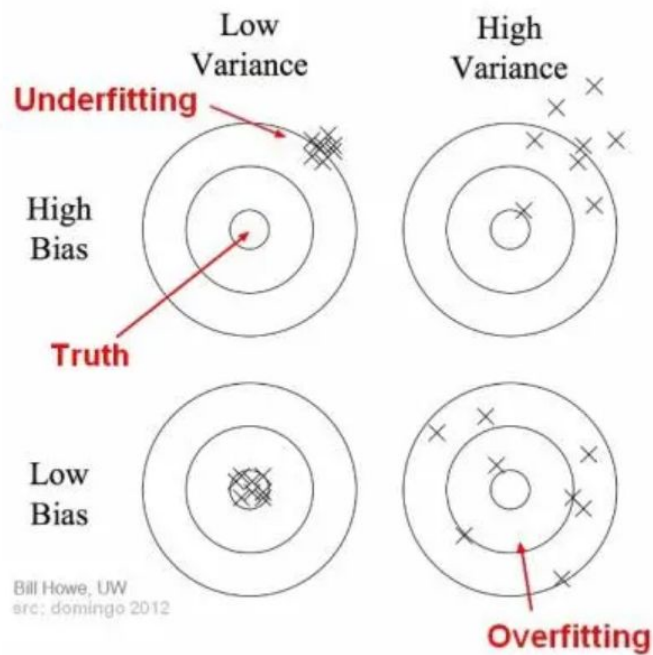
WHAT IS BIAS?

- Bias is the difference between the average prediction of our model and the correct value which we are trying to predict. Model with high bias pays very little attention to the training data and oversimplifies the model. It always leads to high error on training and test data.

WHAT IS VARIANCE?

- Variance is the variability of model prediction for a given data point or a value which tells us spread of our data. Model with high variance pays a lot of attention to training data and does not generalize on the data which it hasn't seen before. As a result, such models perform very well on training data but has high error rates on test data.

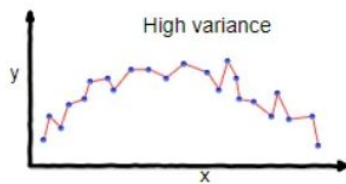
BIAS – VARIANCE



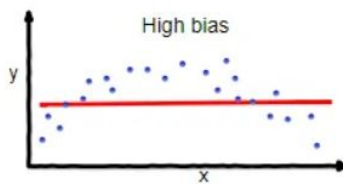
...

- In supervised learning, **underfitting** happens when a model unable to capture the underlying pattern of the data. These models usually have high bias and low variance. It happens when we have very less amount of data to build an accurate model or when we try to build a linear model with a nonlinear data. Also, these kind of models are very simple to capture the complex patterns in data like Linear and logistic regression.
- In supervised learning, **overfitting** happens when our model captures the noise along with the underlying pattern in data. It happens when we train our model a lot over noisy dataset. These models have low bias and high variance. These models are very complex like Decision trees which are prone to overfitting.

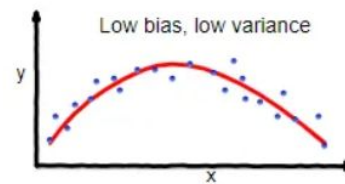
...



overfitting



underfitting



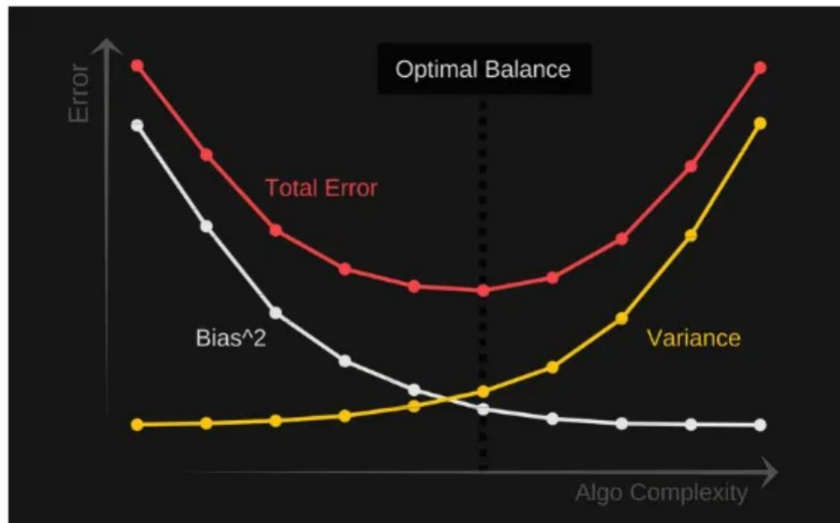
Good balance

In the case of Stochastic Gradient Descent, we update the parameters after every single observation and we know that every time the weights are updated it is known as an iteration.

WHAT IS BIAS VARIANCE TRADEOFF?

- If our model is too simple and has very few parameters then it may have high bias and low variance. On the other hand if our model has large number of parameters then it's going to have high variance and low bias. So we need to find the right/good balance without overfitting and underfitting the data.
- This tradeoff in complexity is why there is a tradeoff between bias and variance. An algorithm can't be more complex and less complex at the same time.

- **Total Error**
- To build a good model, we need to find a good balance between bias and variance such that it minimizes the total error.
- An optimal balance of bias and variance would never overfit or underfit the model.



$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

REGULARIZATION METHODS:

- L1, L2 regularization, Parameter sharing, Dropout, Weight Decay, Batch normalization, Early stopping, Data Augmentation, Adding noise to input and output
- *An effective regularize will be the one that makes the best trade between bias and variance, and the end-product of the tradeoff should be a significant reduction in variance at minimum expense to bias.*
- In simpler terms, this would mean low variance without immensely increasing the bias value.

- We consider two scenarios:
 1. The true data-generating process/function: F_1 , which created the dataset
 2. Creating a generating process/function: F_2 that mimics F_1 but also explores other possible generating scenarios/functions
- The work of regularization techniques is to help take our model from F_2 to F_1 without overly complicating F_2 . Deep learning algorithms are mostly used in more complicated domains like images, audio, text sequences or simulating complex decision making tasks. **The True data-generation process: F_1 is almost impossible to be correctly mapped, hence with regularization, we aim to bring our model with F_2 function as close as possible to the original F_1 function.**

What is L1 and L2 Regularization?

- L1 and L2 regularization are techniques used to prevent overfitting in machine learning models, particularly in deep learning. They add a penalty term to the loss function during training, which discourages the model from learning overly complex representations of the training data.
- L1 regularization (Lasso) promotes sparsity, driving some weights to exactly zero, while L2 regularization (Ridge) shrinks weights towards zero without necessarily making them zero, making models more robust

L1 Regularization (Lasso):

How it works:

L1 regularization adds a penalty term to the loss function proportional to the absolute value of the model's weights. This term is calculated as the sum of the absolute values of all weights in the model. 

Effect on model:

L1 regularization tends to produce sparse models, where many weights are driven to zero. This can be useful for feature selection, as features with zero weights effectively become irrelevant. 

Mathematical representation:

If w_i represents the i -th weight, the L1 regularization term is $\lambda \sum_i |w_i|$, where λ is the regularization strength, controlling the amount of penalty. 

L2 Regularization (Ridge):

How it works:

L2 regularization adds a penalty term to the loss function proportional to the square of the model's weights. This is calculated as the sum of the squares of all weights in the model.

Effect on model:

L2 regularization shrinks weights towards zero, but they rarely reach exactly zero. It's effective in reducing the impact of individual features and preventing overfitting by making the model more generalizable.

Mathematical representation:

If w_i represents the i -th weight, the L2 regularization term is $\lambda \sum_i w_i^2$.

L1 AND L2 REGULARIZATION

Regularization adds the penalty as model complexity increases. The regularization parameter (lambda) penalizes all the parameters except intercept so that the model generalizes the data and won't overfit.

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^N |w_i|$$

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

$\min_{\theta} J(\theta)$

```
from keras import regularizers
model.add(Dense(64, input_dim=64,
                kernel_regularizer=regularizers.l2(0.01),
                activity_regularizer=regularizers.l1(0.01)))
```


Presentation title

L1 Regularisation	L2 Regularisation
Sum of absolute value of weights	Sum of square of weights
Sparse solution is the outcome	Non-sparse (more segregated) solution
Multiple solutions are possible	Only one solution
Built-in feature selection in the penalty term	No specific feature selection mechanism
Robust to outliers	Not robust to outliers due to square term
Used in datasets with sparse features	Used in dataset with complex features

MORE REGULARIZATION TECHNIQUES

- Regularization is a technique to prevent overfitting in neural networks, which occurs when the model learns the training data too well and fails to generalize to new data.

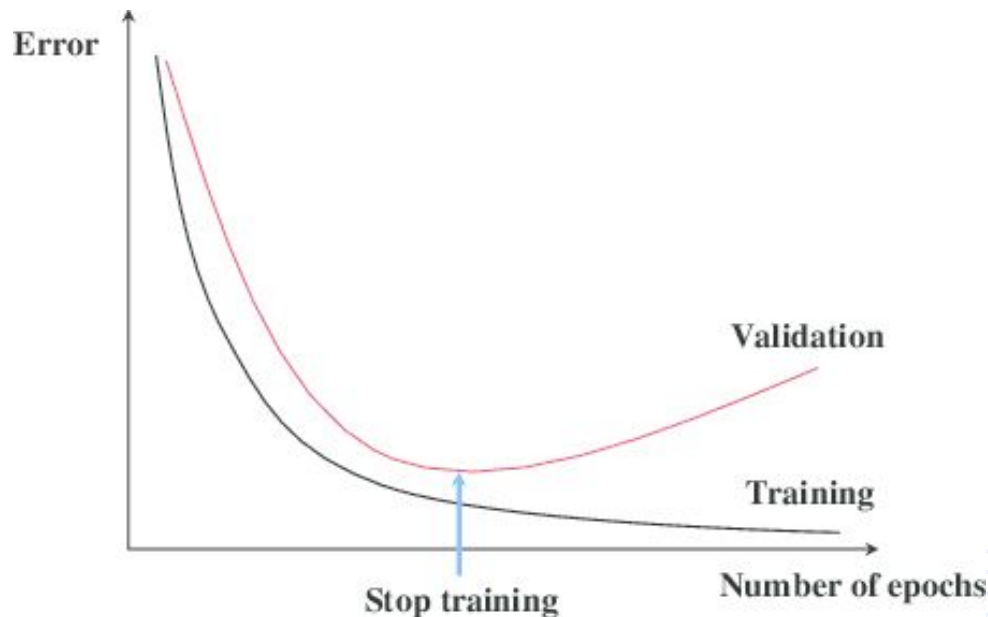
EARLY STOPPING

It is perhaps the simplest regularization strategy.

As the name suggests in early stopping, we stop the training early.

By stopping the training of our model early, we can prevent our model from overfitting.

For instance, our model might keep reducing its loss in the training data and keep increasing its loss in the validation data. This is a sign of overfitting.

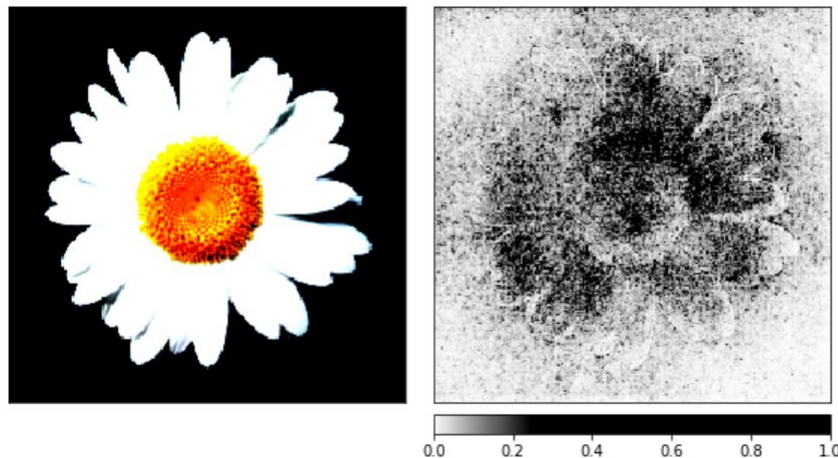


```
keras.callbacks.callbacks.EarlyStopping(monitor='val_loss', min_delta=0,  
patience=0)
```

ADDING NOISE TO INPUT AND OUTPUT

A common approach is to add noise to the input data of the network during the training procedure.

However, in some situations adding noise to the hidden units or to the network weights also leads to improved generalization performance, thus reducing the effect of overfitting.



`keras.layers.GaussianNoise()`

[Fooling Neural Networks with Noise – Kudelski Security Research](#)

DROPOUT

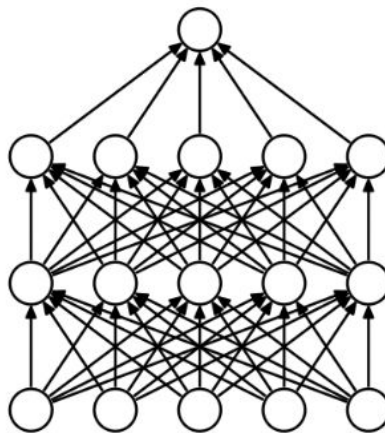
Dropout is an effective way of regularizing neural network, which can be applied to the output of some of the network layers to avoid overfitting.

The key idea here is to randomly drop some proportion of neurons along with their connections from the neural network during training.

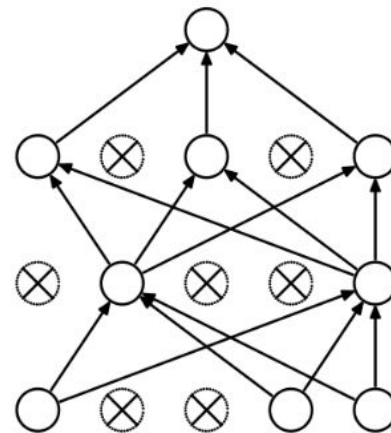
The dropout layer has something called a Dropout rate(p), which ranges between 0 and 1 (both included).

If n is the number of neurons in the hidden layer and p is the dropout rate, then only $(p \cdot n)$ neurons will be active at each given time.

We randomly drop the neurons in the hidden layer based on the dropout rate.



(a) Standard Neural Net



(b) After applying dropout.

`keras.layers.Dropout()`

DATA

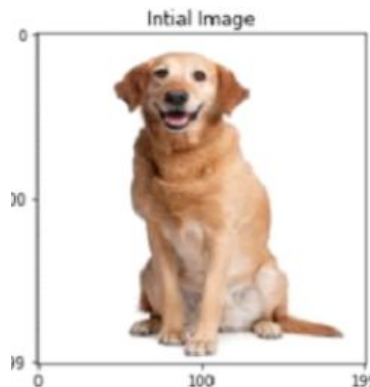
AUGMENTATION

The neural network requires a lot of data to train, and our model might start to overfit if our training data is too small.

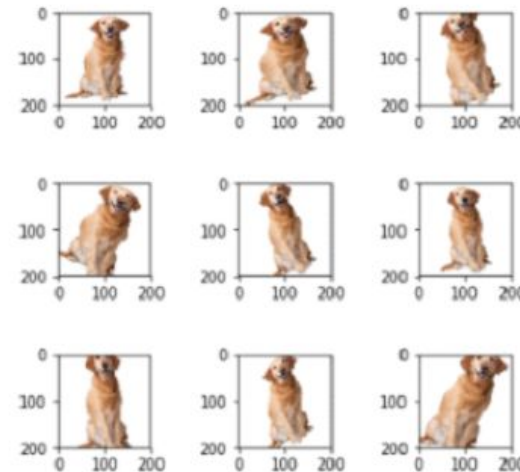
Data augmentation is a regularization technique that aims to combat this by increasing the size of the training set artificially.

Data augmentation depends on the type of data. For some types of data, it may be easy to create artificial data like images, and for some data like the text, it may not be very easy.

In the case of image data, we can artificially create new images by slightly rotate, resize, flip the image horizontally or vertically, skewing the image, etcetera.



Augmented Images



```
keras.preprocessing.image.ImageDataGenerator(featurewise_center=False, samplewise_center=False,  
featurewise_std_normalization=False, \n  
samplewise_std_normalization=False, zca_whitening=False, zca_epsilon=1e-06, rotation_range=0, \nwidth_shift_range=0.0, height_shift_range=0.0, brightness_range=None, shear_range=0.0, \n  
zoom_range=0.0, \n  
channel_shift_range=0.0, fill_mode='nearest', cval=0.0, horizontal_flip=False, vertical_flip=False, \nrescale=None, preprocessing_function=None, data_format='channels_last', validation_split=0.0, \ninterpolation_order=1, dtype='float32')
```

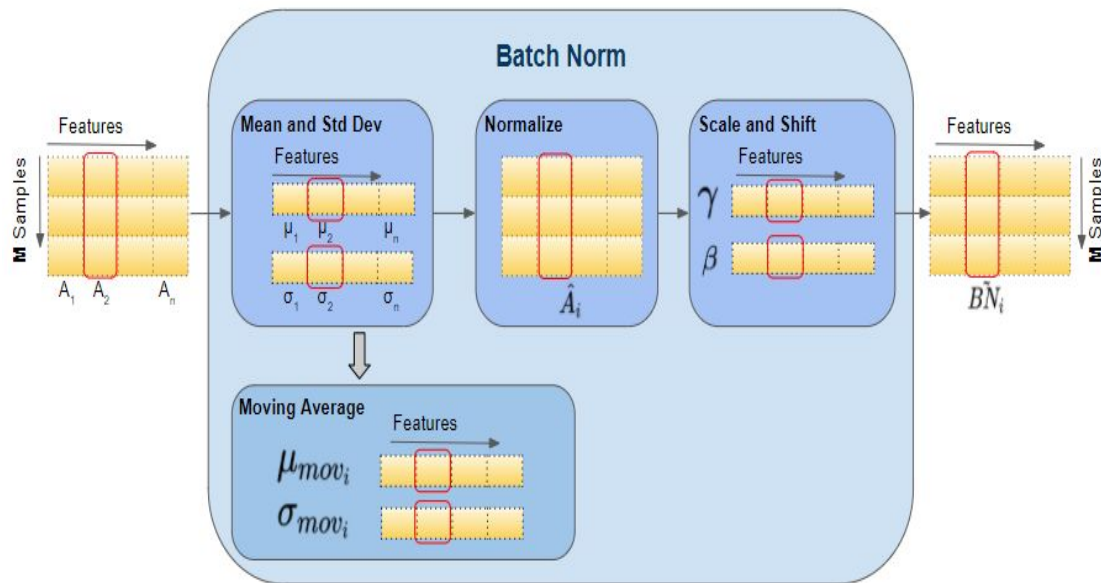
BATCH

Presentation title

NORMALIZATION

Batch normalization normalizes the output from a layer with zero mean and unit variance. In doing so, the input distribution of the data per batch has less effect on the network.

Batch normalization also acts as a regularizer that prevents the model from overfitting. For this reason, in some cases, they are used instead of dropout layers.



`keras.layers.BatchNormalization()`

SUMMARY

EARLY STOPPING: As the name suggests in early stopping, we stop the training early. By stopping the training of our model early, we can prevent our model from overfitting.

INJECT NOISE: In this technique, we'll add Gaussian noise to the network.

DROPOUT: The key idea here is to randomly drop some proportion of neurons along with their connections from the neural network during training.

DATA AUGMENTATION: In this technique, we increase the size of the training set artificially.

L1 AND L2 REGULARIZER: Imposing certain constraints on the model weights and control how much the weights can grow or shrink in the network during training.

BATCH NORMALIZATION: Normalizing the output from a layer with zero mean and unit variance.

IMPORTANT QUESTIONS ON MODULE 2

1. Explain the learning factors for neural networks.
2. What is Activation Function? Explain the significance of activation function
3. Explain different (L1, L2 and Dropout) Regularization Methods.
4. Explain dropout. How does it solve the problem of overfitting?
5. Explain Dropout method and its benefits.
6. What is regularization? Compare L1 and L2 regularization methods.
7. What is an activation function? Describe any two activation functions with advantages and disadvantages.
8. Explain the difference between: RELU, LRELU AND SIGMOID ACTIVATION FUNCTIONS
9. What is activation function? Explain the significance of activation function in neural networks?
10. What is Loss function? Explain Squared Error loss and Cross Entropy loss function
11. What are different types Gradient Descent Algorithms. Explain any three of them
12. Explain the process of Choosing output function and loss function in neural networks.
13. Explain stochastic gradient descent algorithm and momentum based gradient descent algorithm.
14. Explain in detail Gradient Descent Algorithm in deep learning.
15. Explain the following optimization algorithms:
 - a. Nesterov Accelerated GD
 - b. AdaGrad
 - c. Adam
 - d. RMSProp
16. Explain early stopping, Batch normalization, Data augmentation.



THANK YOU